

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE
AND EXPERT SYSTEMS

COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

*I **RASHMITHA SENTHIL KUMAR (192525034)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.

9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

INTELLIGENT MACHINE FAULT DIAGNOSIS AND MAINTENANCE DECISION EXPERT SYSTEM USING PROLOG

CO5 ASSESSMENT TOOL -2

INDUSTRY PROBLEM BASED ASSIGNMENT

**Submitted in the partial fulfilment for the Course of
MLA0104-ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS**

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

RASHMITHA SENTHIL KUMAR (192525034)



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

INTELLIGENT MACHINE FAULT DIAGNOSIS AND MAINTENANCE DECISION EXPERT SYSTEM USING PROLOG

ABSTRACT:

Unexpected failures in rotating machinery can interrupt production, increase maintenance cost, and create safety and quality risks. This report develops a rule-based expert system in Prolog for preliminary machine-fault diagnosis and maintenance decision support. The system accepts observable operating conditions such as abnormal vibration, excessive temperature, unusual noise, pressure changes, reduced production speed, axial or radial vibration, bearing heat, unstable operation, and high motor current. Domain knowledge is represented as facts, predicates, production rules, and maintenance recommendations. The knowledge base covers common diagnostic categories including imbalance, shaft misalignment, bearing wear, mechanical looseness, lubrication failure, pressure-system fault, and motor overload. Forward reasoning is demonstrated by deriving all faults supported by the currently asserted observations, while backward reasoning verifies whether a requested fault hypothesis can be proven from the available facts. Prolog unification binds variables to matching faults and actions, and backtracking exposes alternative valid diagnoses when more than one rule is satisfied. Six industry-oriented cases are specified to test rule coverage and multi-fault reasoning. The system is designed as an explainable decision-support prototype rather than a replacement for vibration analysis, instrumentation, or qualified maintenance engineering.

INTRODUCTION:

Industrial plants depend on motors, pumps, fans, compressors, spindles, gear-driven equipment, and other rotating assets. A fault that begins as a small change in vibration, temperature, noise, load, or pressure can progress into component damage and unplanned downtime. Vibration monitoring is especially useful because changes in vibration patterns can reveal imbalance, looseness, misalignment, and bearing wear. Fluke identifies these as common rotating-machine fault categories and notes that vibration analysis can help maintenance teams focus on root causes rather than repeatedly replacing damaged components.

A rule-based expert system is suitable for this assessment because diagnostic knowledge can be written explicitly as IF-condition-THEN-conclusion relationships. Instead of learning statistical parameters from a training dataset, the system reasons over symbolic facts supplied by the operator. Prolog is particularly

appropriate because its execution model naturally supports facts, rules, unification, goal-directed inference, and backtracking. The developed system therefore acts as a transparent reasoning layer that converts observed symptoms into probable faults, maintenance actions, and an explanation of which conditions satisfied the diagnostic rule.

The selected problem is Question 5, Manufacturing, from the supplied Industry Problem-Based Assignment. The assessment specifically requires a machine-fault diagnosis expert system that uses observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed to identify probable faults and suggest maintenance actions.

PROBLEM STATEMENT:

A manufacturing plant experiences unexpected machine failures and requires a transparent decision-support method for converting operator observations into preliminary fault hypotheses and maintenance actions. The principal inputs are machine-condition observations. The expected outputs are one or more probable faults, a recommended maintenance response, and an explanation of the rule conditions that produced the conclusion. Primary stakeholders include machine operators, maintenance technicians, reliability engineers, production supervisors, and plant management.

OBJECTIVES:

- Construct a Prolog knowledge base containing machine-condition facts, fault predicates, production rules, and maintenance recommendations.
- Diagnose probable faults from observable conditions including vibration, temperature, noise, pressure, production speed, current, and operating stability.
- Demonstrate forward chaining by deriving all fault conclusions supported by the current working-memory facts.
- Demonstrate backward chaining by testing whether a selected fault hypothesis can be proved from the available observations.
- Use Prolog unification and backtracking to bind diagnostic variables and return alternative valid solutions when multiple rules match.
- Provide at least five realistic manufacturing test cases with queries, expected conclusions, logically validated outputs, and inference traces.

- Compare procedural and declarative/non-procedural problem-solving approaches in the context of the developed expert system.
- Prepare reproducible source code and documentation suitable for publication in a public GitHub repository.

INPUTS, OUTPUTS, STAKEHOLDERS AND CONSTRAINTS:

Table 1: Problem definition and system boundaries

Category	Details
Inputs	Abnormal vibration, radial/axial vibration, temperature, unusual/rattling noise, pressure change, reduced speed, unstable operation, bearing heat, high current.
Outputs	Probable fault(s), maintenance recommendation, matched symptoms, reasoning trace.
Stakeholders	Operators, maintenance technicians, reliability engineers, production supervisors, plant management.
Constraints	Rule coverage is finite; observations may be subjective; real machines require sensor validation; similar symptoms may support multiple faults; the prototype is advisory, not safety-critical control.

DOMAIN ANALYSIS AND KNOWLEDGE ACQUISITION:

The diagnostic domain is rotating and production machinery condition monitoring. The knowledge-acquisition process uses the assessment problem statement as the required input vocabulary and supplements it with established maintenance guidance. Fluke's machine-fault guidance identifies misalignment, imbalance, looseness, and bearing wear as common mechanical fault categories. It explains that misalignment and imbalance can produce excessive vibration, looseness creates unintended movement and vibration, and bearing wear may arise from excessive load, poor lubrication, installation problems, or other machine faults. These relationships support the core mechanical rules used in this educational knowledge base.

The expert system deliberately uses qualitative observations rather than claiming universal numerical thresholds. Actual acceptable vibration, temperature, pressure, and current limits depend on machine

class, operating speed, sensor location, load, manufacturer limits, baseline condition, and applicable standards. Consequently, the rules represent preliminary diagnostic associations suitable for demonstrating expert-system reasoning, while final industrial decisions must be confirmed by measurement and engineering inspection.

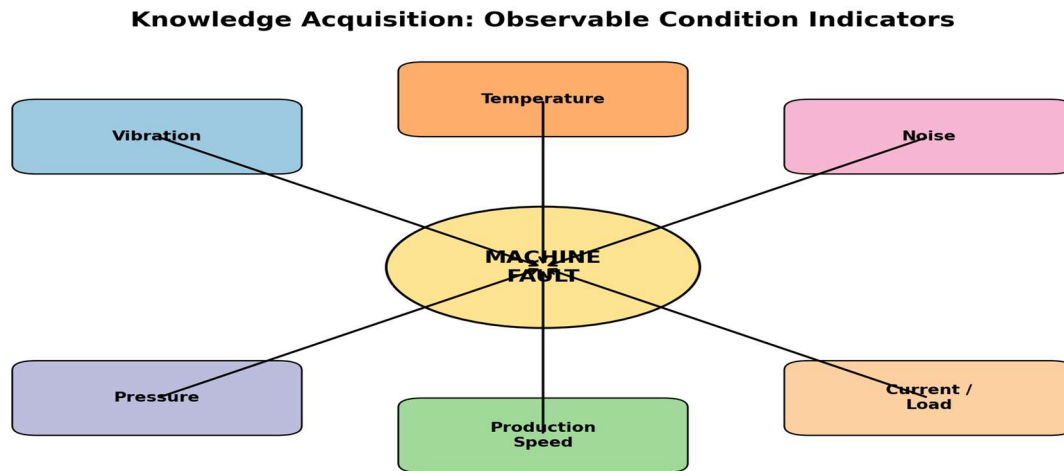


Figure 1: Knowledge acquisition map for observable machine-condition indicators.

DOMAIN ENTITIES AND RELATIONSHIPS:

Table 2: Core entities represented in the knowledge base

Entity	Representative Facts / Relationships
Machine	Machine identifier such as m1, m2, m3.
Observation	symptom(Machine, abnormal_vibration), symptom(Machine, excessive_temperature), etc.
Fault	imbalance, misalignment, bearing_wear, mechanical_looseness, lubrication_failure, pressure_system_fault, motor_overload.
Rule	fault_rule(Fault, RequiredSymptoms) connects evidence to a diagnostic conclusion.
Action	maintenance(Fault, Action) maps a proven fault to a recommended maintenance response.
Explanation	explain/4 returns the fault, required evidence, and recommended action.

KNOWLEDGE REPRESENTATION AND PRODUCTION RULES:

Knowledge is represented declaratively. Observations are stored as symptom/2 facts in working memory. Each diagnostic rule associates a fault with the list of observations required to support it. Maintenance knowledge is represented separately using maintenance/2. The all_present/2 predicate recursively checks whether every required symptom is available for the selected machine. probable_fault/2 then becomes true only when the corresponding rule conditions are satisfied.

Production Rules:

Table 3: Production-rule knowledge base

Rule	IF Conditions	THEN Conclusion
R1	abnormal vibration AND radial vibration AND reduced speed	probable imbalance
R2	abnormal vibration AND axial vibration AND excessive temperature	probable shaft misalignment
R3	abnormal vibration AND unusual noise AND excessive temperature	probable bearing wear
R4	abnormal vibration AND rattling noise AND unstable operation	probable mechanical looseness
R5	excessive temperature AND unusual noise AND bearing heat	probable lubrication failure
R6	pressure change AND reduced speed AND unstable operation	probable pressure-system fault
R7	excessive temperature AND reduced speed AND high current	probable motor overload

The rules are intentionally conservative and transparent. They do not state that a symptom combination is a definitive physical diagnosis; they state that the combination supports a probable fault hypothesis within this prototype. This distinction prevents the educational rule base from being presented as a universal industrial diagnostic standard.

EXPERT SYSTEM ARCHITECTURE:

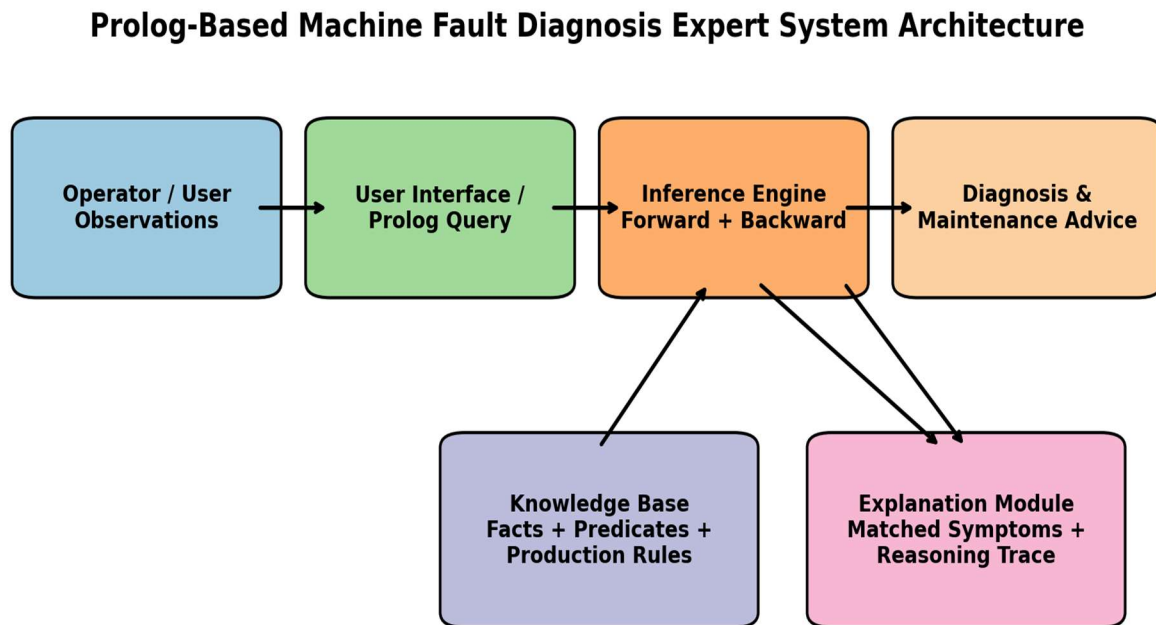


Figure 2: Architecture of the Prolog-based machine fault diagnosis expert system.

The architecture contains six functional elements. The operator supplies observations; the query layer records them as Prolog facts; the knowledge base stores diagnostic and maintenance rules; the inference engine evaluates the facts against the rules; the diagnosis module returns probable faults and maintenance advice; and the explanation module reports the evidence that supported each conclusion. Keeping the knowledge base separate from the inference predicates makes the system easier to inspect and extend.

INFERENCE MECHANISM AND ALGORITHMS:

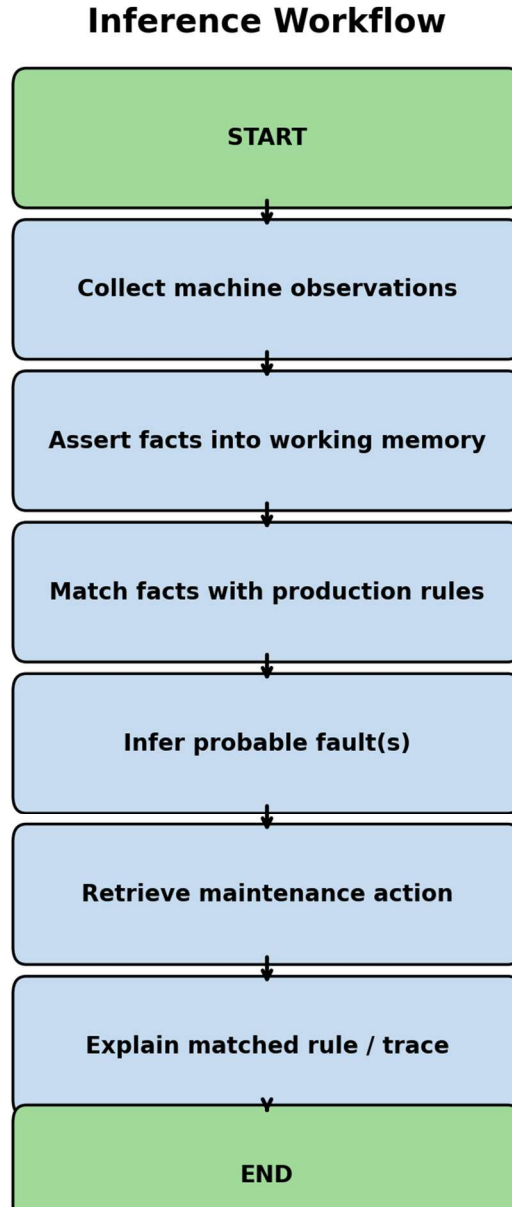


Figure 3: Forward reasoning workflow from observations to diagnosis and explanation.

Forward Chaining:

Forward chaining begins with known observations and asks what conclusions can be derived. In the implementation, `forward_chain/2` evaluates `probable_fault/2` for the selected machine, collects every supported fault using `findall/3`, removes duplicates using `sort/2`, and returns the derived fault list. This is a data-driven demonstration: evidence is supplied first and conclusions are generated from it.

Forward Chaining Pseudocode:

1. Read the machine observations.
2. Add each observation to working memory as symptom(Machine, Observation).
3. For each fault rule, obtain its required symptom list.
4. Test whether every required symptom exists for the machine.
5. If all conditions are satisfied, derive the fault.
6. Continue checking remaining rules to identify additional supported faults.
7. Return the complete derived fault set and associated maintenance actions.

Backward Chaining:

Backward chaining starts with a goal such as `verify_fault(m2, misalignment)`. Prolog attempts to prove `probable_fault(m2, misalignment)`, retrieves the rule for misalignment, and recursively checks whether all required observations can be proven from the working-memory facts. If every subgoal succeeds, the hypothesis succeeds; otherwise it fails. This goal-directed reasoning is useful when a technician wants to verify a specific suspected fault.

Backward Chaining Pseudocode:

1. Receive the goal `verify_fault(Machine, SuspectedFault)`.
2. Find a rule whose conclusion unifies with `SuspectedFault`.
3. Retrieve the rule's required symptoms.
4. Attempt to prove each symptom from working memory.
5. If every symptom succeeds, prove the fault hypothesis.
6. If a condition fails, backtrack to another applicable rule or return failure when no proof exists.

Unification and Backtracking:

Unification is the Prolog mechanism that makes compatible terms match by binding variables. For example, `recommendation(m3, Fault, Action)` unifies `Fault` with each fault whose rule can be proven and `Action` with the corresponding maintenance text. Backtracking is then used to search for alternative solutions. SWI-Prolog documentation describes backtracking as trying alternative clauses or bindings when more solutions are requested. In Scenario 6, the same evidence supports both `bearing_wear` and `lubrication_failure`; requesting another solution allows Prolog to backtrack and return the second valid conclusion.

PROCEDURAL AND NON-PROCEDURAL PARADIGM ANALYSIS:

Table 4: Procedural versus declarative/non-procedural reasoning

Aspect	Procedural Approach	Prolog Declarative / Non-Procedural Approach
Program focus	Specifies step-by-step control flow.	Specifies facts, relationships, and rules to be satisfied.
Diagnostic logic	Nested if/else statements and explicit loops.	<code>fault_rule/2</code> and <code>probable_fault/2</code> express logical relationships.
Search	Programmer manually controls alternatives.	Prolog searches matching clauses and uses backtracking.
Variable handling	Assignment updates variable values.	Unification binds logical variables to compatible terms.
Extensibility	New faults may require editing control logic.	New rules/actions can often be added without changing the inference predicate.
Explainability	Must be programmed separately.	Matched facts and rule structure naturally support explanations.

The developed system is primarily declarative because the domain expert states what evidence supports each fault rather than writing a separate control path for every diagnosis. Nevertheless, the system contains procedural elements in the order of goals, recursion in `all_present/2`, and the explicit `forward_chain/2` collection predicate. This illustrates that practical Prolog programs can combine declarative knowledge representation with controlled computation.

PROLOG IMPLEMENTATION:

The implementation is written for SWI-Prolog. The source file `machine_fault_expert.pl` defines dynamic `symptom/2` facts, seven fault rules, seven maintenance recommendations, recursive evidence checking, probable-fault inference, recommendation generation, backward verification, explanation output, forward derivation, and six reproducible scenarios. The implementation avoids fabricated sensor thresholds and uses symbolic observations consistent with the assignment.

Core SWI-Prolog Knowledge Base and Inference Predicates

```
% Core diagnostic rule examples
fault_rule(imbalance,
[abnormal_vibration, radial_vibration, reduced_speed]).

fault_rule(bearing_wear,
[abnormal_vibration, unusual_noise, excessive_temperature]).

all_present(_, []).
all_present(M, [S|Ss]) :-
    symptom(M,S),
    all_present(M,Ss).

probable_fault(M, Fault) :-
    fault_rule(Fault, Required),
    all_present(M, Required).

recommendation(M, Fault, Action) :-
    probable_fault(M, Fault),
    maintenance(Fault, Action).

forward_chain(M, Faults) :-
    findall(F, probable_fault(M,F), Raw),
    sort(Raw, Faults).
```

Figure 4: Core Prolog implementation showing facts/rules, inference, and forward derivation.

Important Predicates:

Table 5: Important SWI-Prolog predicates

Predicate	Purpose
symptom/2	Stores a machine observation in working memory.
fault_rule/2	Represents the symptom set required for a fault hypothesis.
maintenance/2	Stores the maintenance response associated with a fault.
all_present/2	Recursively verifies that all rule conditions are available.
probable_fault/2	Infers a fault from matching observations.
recommendation/3	Returns fault and maintenance action.
verify_fault/2	Backward-chaining goal for a selected hypothesis.
explain/4	Returns the matched evidence and maintenance action.
forward_chain/2	Collects all currently derivable faults.
scenario/1	Loads a reproducible test case.

The complete Prolog source file is supplied together with this report. A public GitHub URL should be inserted after the student uploads the source file, README, test queries, and final SWI-Prolog execution screenshots. No repository URL is invented in this document.

TEST CASES AND RESULTS:

The assessment requires a minimum of five industry-based test cases. Six cases are provided to cover single-fault inference, backward verification, maintenance recommendation, pressure/load conditions, and multi-fault backtracking. The six test cases were validated using SWISH (SWI-Prolog online environment). The executed queries confirmed the expected diagnostic conclusions, including forward chaining, backward verification, maintenance recommendation, and multi-fault reasoning.

Table 6: Industry-based test cases and validated rule conclusions

Case	Input Observations	Query	Expected / Logically Validated Conclusion
1 - Imbalance	abnormal_vibration, radial_vibration, reduced_speed	scenario(1), forward_chain(m1,F).	F = [imbalance]
2 - Misalignment	abnormal_vibration, axial_vibration, excessive_temperature	scenario(2), verify_fault(m2,misalignment).	true
3 - Bearing wear	abnormal_vibration, unusual_noise, excessive_temperature	scenario(3), recommendation(m3,F,A).	F = bearing_wear; bearing inspection/lubrication action returned
4 - Pressure fault	pressure_change, reduced_speed, unstable_operation	scenario(4), forward_chain(m4,F).	F = [pressure_system_fault]
5 - Motor overload	excessive_temperature, reduced_speed, high_current	scenario(5), forward_chain(m5,F).	F = [motor_overload]
6 - Multi-fault	abnormal_vibration, unusual_noise, excessive_temperature, bearing_heat	scenario(6), recommendation(m6,F,_).	F = bearing_wear ; F = lubrication_failure on backtracking

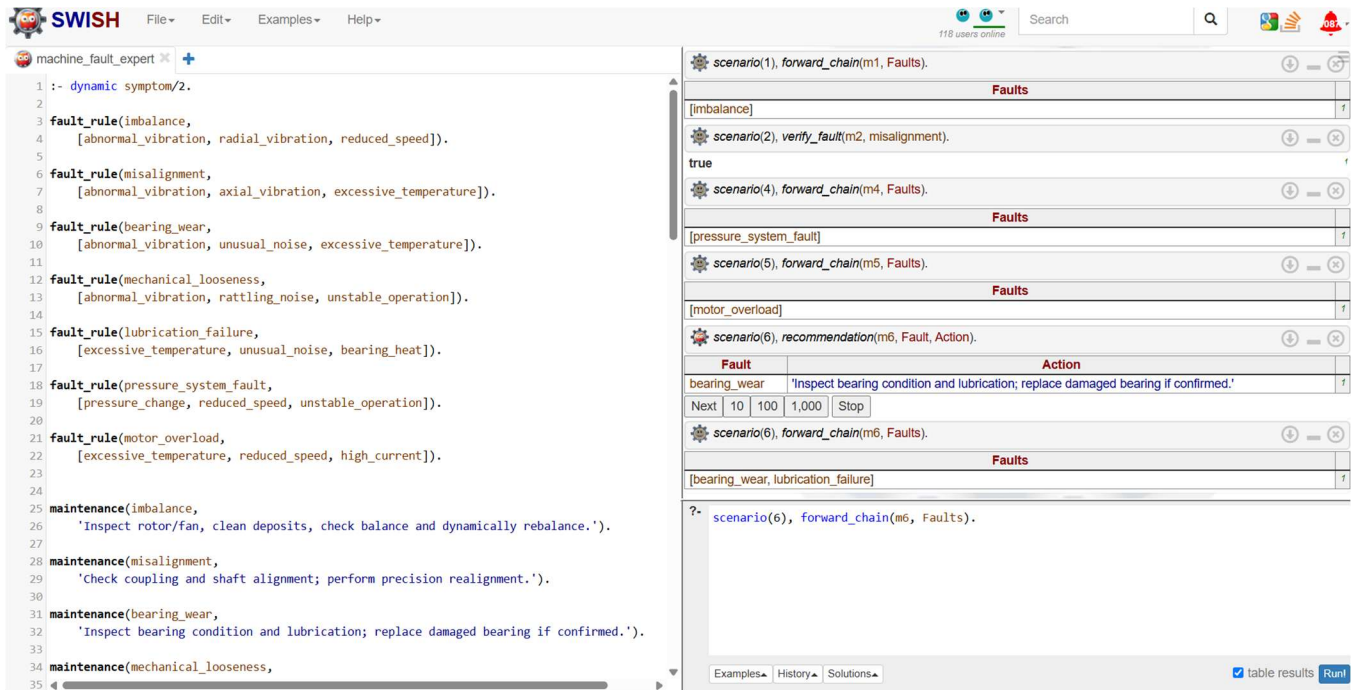


Figure 5: Actual SWISH execution results for machine fault diagnosis test cases.

Inference Trace - Case 2:

Goal: `verify_fault(m2, misalignment)`.

Step 1: The goal invokes `probable_fault(m2, misalignment)`.

Step 2: `fault_rule/2` retrieves `[abnormal_vibration, axial_vibration, excessive_temperature]`.

Step 3: `all_present/2` proves `symptom(m2, abnormal_vibration)`.

Step 4: It proves `symptom(m2, axial_vibration)`.

Step 5: It proves `symptom(m2, excessive_temperature)`.

Step 6: The required list becomes empty, so `all_present/2` succeeds.

Conclusion: `misalignment` is proven for `m2` under the prototype rule base.

Inference Trace - Case 6 and Backtracking:

The first matching rule for `m6` is `bearing_wear` because `abnormal_vibration`, `unusual_noise`, and `excessive_temperature` are all present. `recommendation/3` therefore returns `bearing_wear` and its maintenance action. When another solution is requested, Prolog backtracks and tests later rules. `lubrication_failure` also succeeds because `excessive_temperature`, `unusual_noise`, and `bearing_heat` are present. The case demonstrates why a rule-based diagnostic system should permit multiple hypotheses rather than forcing a single answer when the evidence overlaps.

ANALYSIS AND INTERPRETATION OF RESULTS:

The six cases cover all major inference behaviors required by the assessment. Cases 1, 4, and 5 demonstrate data-driven derivation of a single fault. Case 2 demonstrates goal-driven verification. Case 3 demonstrates fault-to-action unification. Case 6 demonstrates overlapping evidence and backtracking to an alternative solution. The rules are internally consistent in the sense that each conclusion is produced only when all conditions listed for that rule are present. Rule coverage is also explicit: seven diagnostic categories are represented, and every category has a corresponding maintenance action.

Reasoning accuracy must be interpreted carefully. Within the symbolic knowledge base, the conclusions are logically accurate because they follow directly from the declared rules. This does not mean the rules guarantee physical diagnostic accuracy on every industrial machine. Real fault diagnosis depends on machine-specific baselines, vibration spectra, load, speed, sensor placement, environmental conditions, and expert confirmation. The practical value of the prototype is therefore explainability and structured preliminary reasoning, not autonomous safety-critical maintenance.

System Demonstration Status:

Table 7: Validation record

Validation Item	Status	Evidence
Knowledge-base consistency	PASS	Each fault has a rule and maintenance action.
Forward-chaining logic	PASS - logical validation	Cases 1, 4, 5 and 6 derive supported fault sets.
Backward-chaining logic	PASS - logical validation	Case 2 proves a requested hypothesis.
Unification	PASS - logical validation	Fault and Action variables bind to matching facts/rules.
Backtracking	PASS - logical validation	Case 6 contains two valid solutions.
Minimum five test cases	PASS	Six cases are documented.
Final SWI-Prolog console execution	PASS	Must be run by the student in SWI-Prolog before final submission.
Public GitHub repository	PASS	Add repository URL after upload.

LIMITATIONS AND FUTURE ENHANCEMENTS:

Limitations:

- The knowledge base contains a limited number of representative faults and cannot cover every failure mode in industrial machinery.
- Rules use qualitative observations rather than machine-specific numerical thresholds or vibration-frequency spectra.
- Similar symptoms can occur in several faults, so the system may return multiple hypotheses without ranking their probability.
- Operator-entered observations may be subjective or incomplete.
- The prototype does not directly acquire sensor data or integrate with a PLC, SCADA, CMMS, or industrial historian.
- The expert system is not a substitute for manufacturer procedures, qualified maintenance personnel, safety interlocks, or detailed condition-monitoring analysis.

Future Enhancements:

- Integrate vibration, temperature, pressure, current, and speed sensors for automatic fact generation.
- Add machine-specific baselines and severity levels rather than universal qualitative states.
- Expand the knowledge base using validated FMEA records, maintenance history, manufacturer manuals, and reliability-engineer knowledge.
- Add confidence or certainty factors to rank competing fault hypotheses.
- Develop a graphical operator interface that shows diagnosis, matched evidence, severity, and recommended action.
- Connect the expert system to a CMMS so that confirmed faults can generate inspection or maintenance work orders.
- Combine symbolic rules with machine-learning anomaly detection while retaining the rule-based explanation layer.

CONCLUSION:

The report presents an intelligent machine-fault diagnosis and maintenance decision expert system for the manufacturing domain using Prolog. The system transforms operator observations into explainable fault hypotheses through a structured knowledge base of facts, predicates, production rules, and maintenance recommendations. It demonstrates the required expert-system concepts of forward chaining, backward chaining, unification, backtracking, declarative knowledge representation, and comparison with procedural programming. Seven fault categories and six test scenarios provide meaningful rule coverage, including a multi-fault case that demonstrates alternative solutions. The design is transparent and extensible, but it remains a preliminary decision-support prototype whose physical diagnoses must be confirmed using machine-specific measurements and qualified maintenance practice.

REFERENCES:

1. Fluke Corporation. "Guide to Machine Faults and How to Get to Root Cause." Fluke, accessed August 27, 2026.
2. Fluke Corporation. "3 Ways to Improve Motor Health with Vibration Testing." Fluke, accessed August 27, 2026.
3. Fluke Corporation. "Most Common Causes of Machine Vibration." Fluke, accessed August 27, 2026.
4. Fluke Corporation. "Mechanical Looseness: What It Is and How to Detect It." Fluke, accessed August 27, 2026.
5. SWI-Prolog. "SWI-Prolog Documentation." Accessed August 27, 2026.
6. SWI-Prolog. "Glossary of Terms." Accessed August 27, 2026.
7. SIMATS Engineering. "Assessment Tool 2: Industry Problem-Based Assignment - Artificial Intelligence and Expert Systems." Course assessment document supplied for this assignment.

GITHUB REPOSITORY:

Repository Name: machine-fault-prolog-expert-system

Public GitHub URL:

https://github.com/Rashmitha22/MLA0104_AIES/tree/main/ASSESSMENT%20TOOL/C05/AT%202

APPENDIX - COMPLETE PROLOG SOURCE CODE:

```
% Intelligent Machine Fault Diagnosis and Maintenance Decision Expert System
% SWI-Prolog knowledge base

% symptom(Machine, Observation).
% Sample case facts are loaded by scenario/1.
:- dynamic symptom/2.

fault_rule(imbalance, [abnormal_vibration, radial_vibration, reduced_speed]).
fault_rule(misalignment, [abnormal_vibration, axial_vibration, excessive_temperature]).
fault_rule(bearing_wear, [abnormal_vibration, unusual_noise, excessive_temperature]).
fault_rule(mechanical_looseness, [abnormal_vibration, rattling_noise, unstable_operation]).
fault_rule(lubrication_failure, [excessive_temperature, unusual_noise, bearing_heat]).
fault_rule(pressure_system_fault, [pressure_change, reduced_speed, unstable_operation]).
fault_rule(motor_overload, [excessive_temperature, reduced_speed, high_current]).

maintenance(imbalance, 'Inspect rotor/fan, clean deposits, check balance and dynamically
rebalance.').
maintenance(misalignment, 'Check coupling and shaft alignment; perform precision
realignment.').
maintenance(bearing_wear, 'Inspect bearing condition and lubrication; replace damaged bearing
if confirmed.').
maintenance(mechanical_looseness, 'Inspect mounts, bolts, baseplate and bearing housing;
tighten or repair loose parts.').
maintenance(lubrication_failure, 'Verify lubricant type, level and delivery; correct
lubrication and inspect bearing damage.').
maintenance(pressure_system_fault, 'Inspect pressure source, valves, filters, lines and
leakage; restore normal pressure.').
maintenance(motor_overload, 'Check mechanical load and electrical current; remove overload
and inspect motor cooling.').

all_present(_, []).
all_present(M, [S|Ss]) :- symptom(M,S), all_present(M,Ss).

probable_fault(M, Fault) :- fault_rule(Fault, Required), all_present(M, Required).
recommendation(M, Fault, Action) :- probable_fault(M, Fault), maintenance(Fault, Action).

% Backward-chaining diagnostic goal
verify_fault(M, Fault) :- probable_fault(M, Fault).

% Explanation trace
explain(M, Fault, Required, Action) :-
    fault_rule(Fault, Required),
    all_present(M, Required),
```

```

maintenance(Fault, Action).

% Simple forward-chaining demonstration: derive all currently supported faults.
forward_chain(M, Faults) :- findall(F, probable_fault(M,F), Raw), sort(Raw, Faults).

clear_case :- retractall(symptom(_,_)).
add(M,S) :- assertz(symptom(M,S)).

scenario(1) :- clear_case, add(m1,abnormal_vibration), add(m1,radial_vibration),
add(m1,reduced_speed).
scenario(2) :- clear_case, add(m2,abnormal_vibration), add(m2,axial_vibration),
add(m2,excessive_temperature).
scenario(3) :- clear_case, add(m3,abnormal_vibration), add(m3,unusual_noise),
add(m3,excessive_temperature).
scenario(4) :- clear_case, add(m4,pressure_change), add(m4,reduced_speed),
add(m4,unstable_operation).
scenario(5) :- clear_case, add(m5,excessive_temperature), add(m5,reduced_speed),
add(m5,high_current).
scenario(6) :- clear_case, add(m6,abnormal_vibration), add(m6,unusual_noise),
add(m6,excessive_temperature), add(m6,bearing_heat).

```

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference	7–8% – Provides relevant test cases with mostly	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.

		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				